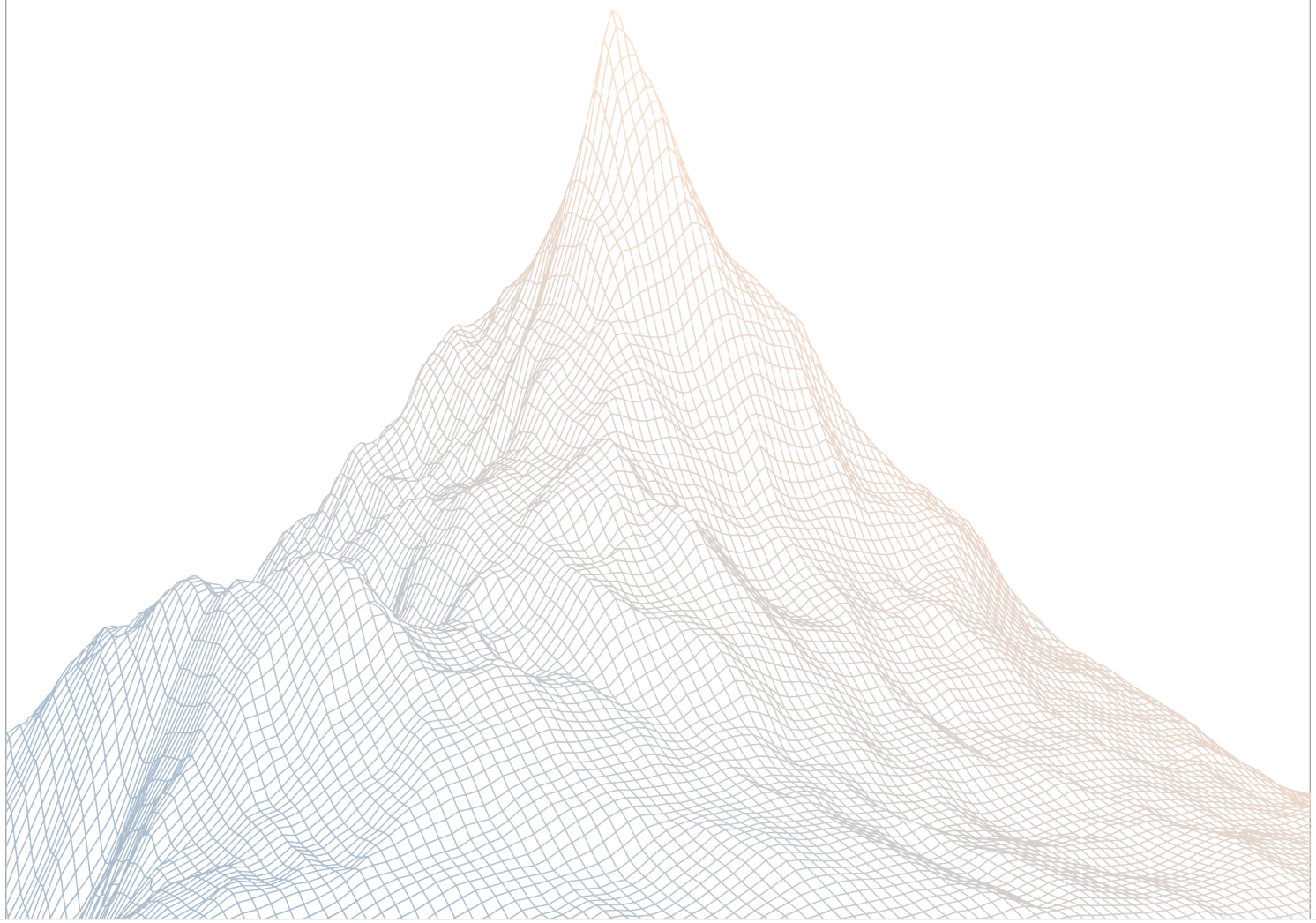
Zenith

MetaDAO

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: February 4th to March 3rd, 2026

AUDITED BY: |||||||
Matte

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About MetaDAO 4

2.2 Scope 4

2.3 Audit Timeline 6

2.4 Issues Found 6

3 Findings Summary 6

4 Findings 9

4.1 High Risk 10

4.2 Medium Risk 11

4.3 Low Risk 27

4.4 Informational 49

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About MetaDAO

A fundraising and governance platform for high-quality founders and their communities.

2.2 Scope

The engagement involved a review of the following targets:

Target	programs
Repository	https://github.com/metaDAOproject/programs
Commit Hash	`ea1879c89d08cdfc47706b0458b2859a7e990f82`
Files	`programs//src//*.rs`

Target	metaDAOproject/programs#414
Repository	https://github.com/metaDAOproject/programs
Commit Hash	`087e0992a7f4b9a3a510d5b27565024fcdb10b65`
Files	`Changes in PR-414`

Target	metaDAOproject/programs#407
Repository	https://github.com/metaDAOproject/programs
Commit Hash	`5beb60de00523ed90b36ecec9e9da244c8f1f27d`
Files	`Changes in PR-407`

Target	metaDAOproject/programs#396
Repository	https://github.com/metaDAOproject/programs
Commit Hash	`ea1879c89d08cdfc47706b0458b2859a7e990f82`
Files	`Changes in PR-396`

2.3 Audit Timeline

February 4, 2026	Audit start
March 3, 2026	Audit end
March 21, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	1
Medium Risk	9
Low Risk	17
Informational	9
Total Issues	**36**

3

Findings Summary

ID	Description	Status
H-1	Bid wall can be abused by non-sale participants	Acknowledged
M-1	price_based_performance_package will be DOSed when the TWAP wraps	Resolved
M-2	performance_package_v2 unlocks may be DOSed by TWAP update saturations	Resolved
M-3	TwapOracle accumulator wrap desynchronization in update_twap	Acknowledged
M-4	Proposal launches with zero liquidity will need to be failed by the admin	Resolved
M-5	CliffLinear allows non-linear cliff payout configuration	Acknowledged
M-6	Squads multisig team addresses cannot be used for optimistic proposals	Resolved
M-7	Bid wall double-counts LP reserves when liquidity is withdrawn to the DAO treasury	Acknowledged
M-8	Delayed performance package initialization may lock its tokens	Resolved
M-9	Performance packages calculate incorrect TWAP prior to the TWAP's start delay	Resolved
L-1	initialize_launch omits additional_tokens_amount from the premine ceiling enforcement	Acknowledged
L-2	Proposals cannot be finalized without trades after the proposal duration	Resolved
L-3	Possible to fund after funds have been returned	Acknowledged
L-4	admin_approve_execute_multisig_proposal permits optimistic-proposal self-deadlock	Resolved
L-5	Proposals may use stale proposal durations	Resolved
L-6	Users can still stake to invalidated transactions	Acknowledged

ID	Description	Status
L-7	A DAO's team can prevent legitimate proposals by continuously sponsoring and launching proposals	Acknowledged
L-8	DAO initialization checks from v07 missing in v06	Resolved
L-9	DAO initialization ignores the user's TWAP start delay for the spot market	Acknowledged
L-10	Performance packages may lose dust amounts due to rounding	Acknowledged
L-11	The execute_change() instructions return rent to the wrong recipient	Acknowledged
L-12	Performance packages' change requests ignore package re-creations	Resolved
L-13	No upper limit on performance package v2's unlock window	Resolved
L-14	Events may be dropped if the SVM truncates its logs	Resolved
L-15	Strict PDA derivations may require client programs to redeploy	Acknowledged
L-16	Bid wall closure may send ATA rent to wrong recipient	Resolved
L-17	Early start_unlock()s by the performance_package_v2 authority may DOS recipient payouts	Resolved
I-1	FundingRecords incompletely Verified.	Resolved
I-2	Accounts need not be marked as mutable	Resolved
I-3	Missing events in some instructions	Resolved
I-4	Squads TransactionMessage compilation does not match Squads SDK	Resolved
I-5	Funding is allowed for one second longer than expected	Resolved

ID	Description	Status
I-6	OracleReader::can_end() reports the wrong answer if record_start() has not been called	Resolved
I-7	Unused instruction accounts	Resolved
I-8	Misleading comments	Acknowledged
I-9	Unused account fields waste rent	Acknowledged

4

Findings

4.1 High Risk

A total of 1 high risk findings were identified.

[H-1] Bid wall can be abused by non-sale participants

SEVERITY: High	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- bid_wall/src/instructions/sell_tokens.rs
- v07_launchpad/src/instructions/claim_additional_token_allocation.rs
- v07_launchpad/src/instructions/complete_launch.rs
- performance_package_v2/src/instructions/initialize_performance_package.rs

Description:

The bid wall attempts to act as buyer of last resort of project tokens, at the participants' launch price, so there is a mechanism for sale buyers to exit fairly. The price sellers to the bid wall get is based on the amount of tokens that are distributed to sale participants at launch, but does not take into account that there may be more tokens than `TOKENS_TO_PARTICIPANTS`, including tokens minted by the DAO, tokens minted by the v2 performance package, and the "additional tokens" minted during the launch, which are immediately claimable. A recipient of such tokens, especially "additional tokens" recipients, can immediately sell their tokens to the bid wall, draining its remaining `quote_amount`, leaving none for the actual participants. If the dilution their allocation implies is large enough, they can buy back the tokens from the Futarchy and Meteora AMMs, locking in a risk-free profit.

Recommendations:

Lock up the "additional tokens" allocation until after the bid wall expires, and don't transfer minting authority to the DAO until the bid wall expires.

MetaDAO: Acknowledged. The additional tokens get locked up as part of our launch system. They are there to be a simple bridge towards a series of vesting contracts that can then be set up separately from our launch. This is not the final version of the launch mechanism, and we're aware of this perceived flaw.

4.2 Medium Risk

A total of 9 medium risk findings were identified.

[M-1] `price_based_performance_package` will be DOSed when the TWAP wraps

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- futarchy/src/state/futarchy_amm.rs
- price_based_performance_package/src/instructions/complete_unlock.rs

Description:

The v2 performance package code points out that it's possible for the TWAP to wrap:

```
// Calculate TWAP: (end_value - start_value) / time_delta
// Note: end_value should always be ≥ start_value since aggregator is
    cumulative
// If end_value < start_value, the aggregator wrapped - rare but possible
// wrapping_sub ensures we get the correct difference despite wrapping
let value_delta = end_value.wrapping_sub(*start_value);

let twap = value_delta / time_delta as u128;
```

Unlike the v2 code, the v1 code does not account for this, and does a saturating subtraction:

```
// Calculate TWAP: (current_aggregator - start_aggregator) / time_passed
let aggregator_change = current_aggregator.saturating_sub(start_aggregator);
let twap_price = aggregator_change / time_passed as u128;
```

In such cases, the `twap_price` will be zero, and the performance package unlock will become permanently unlockable, since the `time_passed` will continue to grow, even if the aggregator ends up reaching the wrap point again.

Recommendations:

Copy the logic from v2, and account for multiple overflows (a separate bug)

MetaDAO: Resolved with `@6218db91bd ...` and `@cc4a77739f ...`

Zenith: Verified.

[M-2] `performance_package_v2` unlocks may be DOSed by TWAP update saturations

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- futarchy/src/state/futarchy_amm.rs
- performance_package_v2/src/state/performance_package.rs
- performance_package_v2/src/state/performance_package.rs

Description:

If there is a long period of time between updates, the calculation that computes the increase in the oracle's `aggregator` may be too large to fit in its variable's type. The TWAP code says that this is acceptable because this will cause the aggregator itself to wrap, which itself is an expected behavior:

```
// if this saturates, the aggregator will wrap back to 0, so this value
    doesn't
// really matter. we just can't panic.
let weighted_observation = new_observation.saturating_mul(slot_difference);

oracle.aggregator.wrapping_add(weighted_observation)
```

However, the completion of a performance package assumes that aggregators that wrap do so exactly:

```
// Calculate TWAP: (end_value - start_value) / time_delta
// Note: end_value should always be ≥ start_value since aggregator is
    cumulative
// If end_value < start_value, the aggregator wrapped - rare but possible
// wrapping_sub ensures we get the correct difference despite wrapping
let value_delta = end_value.wrapping_sub(*start_value);

let twap = value_delta / time_delta as u128;
```

For the old performance package code, the maximum length of a TWAP duration was

one year, so in the new code, if there haven't been any new proposals for a year for a token that has a very high value, the `weighted_observation` may saturate, but the `time_delta` will not have contracted, leading to the TWAP over the period to be much lower than expected, leading to the performance package not being unlocked.

In addition to the saturation issue in the in the TWAP code itself, the v2 performance package code itself repeats this bug when it attempts to project the aggregator's value to the present:

```
let effective_aggregator = aggregator.wrapping_add(
    last_observation.saturating_mul(time_since_update));
```

Recommendations:

If `weighted_observation` wraps, it may mean that the aggregator needs to wrap multiple times, so it's not enough to widen `weighted_observation`. The TWAP will need a counter for how many times it has wrapped, and clients will need to refer to it when determining the TWAP price. To address the projection issue, projection should be added to the TWAP code itself.

MetaDAO: Resolved with `@44118b6d5b ...` We've changed `saturating_mul()` to `wrapping_mul()` to conserve wrapping behavior for one wrap. We acknowledge that it only doesn't work well when things wrap more than once, if we assume that the values we're subtracting are monotonically increasing, which they should be, but ultimately accept the fact that we've got a system that will work just fine in practice.

Zenith: Verified. Multiple wraps causing problems with the fetching of a period's TWAP is acknowledged.

[M-3] `TwapOracle` accumulator wrap desynchronization in `update_twap`

SEVERITY: Medium	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- launch_proposal.rs
- futarchy_amm.rs
- futarchy_amm.rs
- finalize_proposal.rs

Description

Before `finalize_proposal`, each pool must preserve a monotonic cumulative observation under all update sequences; after `ProposalState::Pending → Passed/Failed`, the pass/fail comparison should reflect market trajectory, not `u128` rollover artifacts.

I traced `conditional_swap -> PoolState::swap -> update_twap` in `futarchy_amm.rs`, then followed the threshold check in `finalize_proposal.rs`. `launch_proposal` starts `pass` and `fail` with equal reserves and fresh oracles, but the pools diverge once order flow differs, and each oracle stores cumulative observation with modular addition (modulo arithmetic is unforgiving). Wrap accounting is missing.

```
let effective_last_updated_timestamp =
    oracle.last_updated_timestamp.max(twap_start_timestamp);

let slot_difference: u128 = (current_timestamp -
    effective_last_updated_timestamp)
    .try_into()
    .unwrap();

let weighted_observation = new_observation.saturating_mul(slot_difference);

oracle.aggregator.wrapping_add(weighted_observation)
```

A concrete example shows the invariant break: true cumulative values can be `pass = 260` and `fail = 170`, so `260 > 170`, but with modulo-100 storage the on-chain lows become `60` and `70`, so comparison flips to `60 < 70`. `get_twap` divides only stored low bits by elapsed

seconds, and `finalize_proposal` uses those outputs directly for pass/fail thresholding, so earlier wrap on one side can bias governance outcome even when economic signal pointed the other way.

Impact has two modes. First, settlement bias appears when pass and fail wrap at different times, producing distorted relative TWAPs. Second, exact wrap-to-zero can trigger `require_neq!(self.oracle.aggregator, 0)`, which blocks finalization until another qualifying update arrives after proposal duration, creating liveness friction and a denial path for timely resolution.

Recommendation

Add explicit wrap counting to `TwapOracle`, increment on overflow in `update_twap`, and compute TWAP from an extended accumulator (`wraps * 2^128 + aggregator_low`) instead of low bits alone. I would also enforce a DAO-parameter bound so maximum expected cumulative growth during proposal lifetime cannot approach overflow under worst-case update cadence.

```
diff --git a/programs/futarchy/src/state/futarchy_amm.rs
    b/programs/futarchy/src/state/futarchy_amm.rs

 pub struct TwapOracle {
     pub aggregator: u128,
    pub aggregator_wraps: u64,

         Self {

             aggregator: 0,
            aggregator_wraps: 0,

            oracle.aggregator.wrapping_add(weighted_observation)
            let (next, did_wrap) = oracle.aggregator.overflowing_add(
                weighted_observation);
            if did_wrap {

    oracle.aggregator_wraps = oracle.aggregator_wraps.saturating_
                    add(1);
            }
            next

         let new_oracle = TwapOracle {

             aggregator: new_aggregator,
```

```
        aggregator_wraps: oracle.aggregator_wraps,

    require_neq!(self.oracle.aggregator, 0);
    Ok(self.oracle.aggregator / seconds_passed)
    // reconstruct extended accumulator from high wraps + low bits
    let full = (U256::from(self.oracle.aggregator_wraps) << 128)
        | U256::from(self.oracle.aggregator);
    require!(full > U256::ZERO, FutarchyError::MarketsTooYoung);
    Ok((full / U256::from(seconds_passed)).as_u128())
```

MetaDAO: Acknowledged. We don't (and will never) have DAOs whose proposals last 213 days AND their price is constantly u64::MAX. And I think we can all agree that would be an absurd configuration to begin with. So, while valuable from a theoretical standpoint, I'd wager that this is not an issue.

[M-4] Proposal launches with zero liquidity will need to be failed by the admin

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- futarchy/src/instructions/launch_proposal.rs
- futarchy/src/state/futarchy_amm.rs
- futarchy/src/instructions/finalize_proposal.rs
- v07_launchpad/src/instructions/complete_launch.rs
- v06_launchpad/src/instructions/complete_launch.rs

Description:

When launching a proposal, half of the token reserves from the spot market are used to act as the reserves of the Futarchy pass/fail pools. If half of the reserves ends up being equal to zero for either of the tokens, the AMM will reject all trades, which means the call made to calculate the TWAP in order to determine whether the launched proposal passes or fails, will fail, leaving the DAO in a state where no proposal can pass/launch unless an admin calls `admin_cancel_proposal()` on the existing one, causing it to become failed.

Recommendations:

Require `min_base_futarchic_liquidity` to be at least as large as `base_to_stake`, and `min_quote_futarchic_liquidity` should have at least the equivalent amount when converted from the base amount. There should also be a hard-coded minimum that ensures there will be enough liquidity to perform swaps after the TWAP start delay, and after the proposal's duration. The TWAP code should also be modified to project the last value rather than requiring a trade.

MetaDAO: Resolved with `@554bd9b044 ...` Requiring `min_base_futarchic_liquidity` to be at least as large as `base_to_stake` could potentially brick proposal creation, as at most 20% of the total liquidity is ever present in Futarchy AMM.

Zenith: Verified.

[M-5] `CliffLinear` allows non-linear cliff payout configuration

SEVERITY: Medium	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- performance_package.rs
- performance_package.rs

Description

The `CliffLinear` schedule accepts any `cliff_amount ≤ total_amount`, so the cliff payout is not constrained to the amount that would be earned by a standard linear-vesting-with-cliff curve from `start_value` to `cliff_value`.

I traced initialization/change validation to `RewardFunction::validate` and payout logic to `RewardFunction::calculate`. Validation checks only ordering (`start ≤ cliff ≤ end`) and cap (`cliff_amount ≤ total_amount`), but does not enforce a specific relationship between `cliff_amount` and elapsed progress from `start_value` to `cliff_value`.

```
require!(
    start_value ≤ cliff_value && cliff_value ≤ end_value,
    PerformancePackageError::InvalidVestingSchedule
);

require!(
    cliff_amount ≤ total_amount,
    PerformancePackageError::InvalidVestingSchedule
);
```

The resulting payout shape can diverge from expected "earned-so-far at cliff" behavior. Example: `start=0`, `cliff=25`, `end=100`, `total=1,000`. Standard linear-at-cliff implies `250` earned at cliff. If configured with `cliff_amount=100`, recipient is underpaid at cliff and post-cliff slope increases; if configured with `cliff_amount=600`, recipient is overpaid at cliff and post-cliff slope decreases. Both converge to total 1,000 by `end`, but early termination outcomes differ materially from standard expectations.

This is primarily a specification/configuration risk, not unauthorized minting by an external attacker. However, in governance or compensation contexts, mismatched vesting semantics can still produce high-impact disputes or unintentionally front-loaded payouts.

Recommendation

If the intended product behavior is standard linear vesting with cliff gating, derive `cliff_amount` from schedule geometry rather than accepting it as an unconstrained input. A minimal guard is to enforce `cliff_amount == linear_earned_at_cliff` in `validate`.

```
diff --git
    a/programs/performance_package_v2/src/state/performance_package.rs
    b/programs/performance_package_v2/src/state/performance_package.rs

            RewardFunction::CliffLinear {
                start_value,
                cliff_value,
                end_value,
                cliff_amount,
                total_amount,
            } => {

                require!(
                    cliff_amount <= total_amount,
                    PerformancePackageError::InvalidVestingSchedule
                );

                // Enforce standard linear vesting semantics at the cliff.
                // earned_at_cliff = total_amount * (cliff_value - start_
                    value) / (end_value - start_value)
                if end_value > start_value {
                    let expected = ((cliff_value - start_value) * (*total_
                        amount as u128))
                        / (end_value - start_value);
                    require_eq!(
                        *cliff_amount,
                        expected as u64,
                        PerformancePackageError::InvalidVestingSchedule
                    );
                } else {

    // Degenerate schedule: everything unlocks at one point.
                    require_eq!(
                        *cliff_amount,
                        *total_amount,
                        PerformancePackageError::InvalidVestingSchedule
```

```
                );
            }
        }
```

MetaDAO: Acknowledged. The intended way is actually to have a cliff part and a linear part, with the cliff NOT NECESSARILY being "earned so far as part of the linear vest". What we do with the configuration of this from there on out is really up to how DAOs decide to structure their payouts. Of course, if we decide internally to enforce these semantics, we will apply this.

[M-6] Squads multisig team addresses cannot be used for optimistic proposals

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- futarchy/src/instructions/initiate_vault_spend_optimistic_proposal.rs
- futarchy/src/instructions/initiate_vault_spend_optimistic_proposal.rs

Description:

In order to ensure that an optimistic spend only has the exact transfer being requested, and nothing else, the instruction itself builds the transfer, compiles it, creates the vault transaction, and creates the proposal. The vault transaction and proposal creation steps are Squads instructions, so if the `initiate_vault_spend_optimistic_proposal()` were being executed by a `team_address` that itself was a Squads instruction execution, the operation would fail because Solana does not allow non-self-reentrancy.

Recommendations:

As is done with normal DAO proposals, have the permissionless account create the Squads proposal, but have this ix verify its contents.

MetaDAO: Resolved with `@d31f514dc0 ...`, `@3c85ce4fa9 ...`, `@8717302614 ...` and `@4337ae82ba ...`

Zenith: Verified.

[M-7] Bid wall double-counts LP reserves when liquidity is withdrawn to the DAO treasury

SEVERITY: Medium	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- bid_wall/src/instructions/sell_tokens.rs
- v07_launchpad/src/instructions/complete_launch.rs

Description:

In order to ensure that users have a fair exit option, the bid wall prices NAV as though the DAO is valued according to the initial quote reserves provided as liquidity, the amount of quote tokens in the bid wall, and the value of the DAO's quote tokens in its treasury. Including the DAO's treasury allows the NAV to adjust based on spending done by the DAO, but does not consider the fact that the DAO is the owner of the LP position associated with the Futarchy liquidity position, and may liquidate that position and transfer the tokens to its treasury. If the DAO does this, the initial quote reserves in the calculation are not adjusted, and instead the value is essentially double-counted. This increases the bid wall's NAV such that anyone that sells to the bid wall when this happens, will get many more quote tokens than they otherwise would be able to extract from selling on the open market.

Recommendations:

Transfer the LP shares to the bid wall during bid wall initialization, and send them back to the DAO when the bid wall is closed or cancelled

MetaDAO: Acknowledged, though we would actively prevent such behavior during the first 90 days of the DAO existing, which is also the duration of the bid wall. Won't fix for now, as it's not a realistic issue for now.

[M-8] Delayed performance package initialization may lock its tokens

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- v07_launchpad/src/instructions/initialize_performance_package.rs
- price_based_performance_package/src/instructions/initialize_performance_package.rs

Description:

During launch completion, if there is a v1 performance package configured, the v7 Launch mints the package's tokens to its vault so that later initialization of the package can pull them to its own vault.

If the performance package isn't initialized until after the minimum 18-month lockup, the initialization checks that require the current timestamp to be before the lockup period will fail, blocking the initialization. The extra tokens minted for the package will remain stuck in the launch program's vault.

Recommendations:

Do not require the unlock timestamp to be in the future during performance package initialization

MetaDAO: Resolved with `@8f8b090921 ...`

Zenith: Verified.

[M-9] Performance packages calculate incorrect TWAP prior to the TWAP's start delay

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- price_based_performance_package/src/instructions/start_unlock.rs
- futarchy/src/state/futarchy_amm.rs
- v06_launchpad/src/instructions/complete_launch.rs
- v06_launchpad/src/instructions/initialize_launch.rs
- performance_package_v2/src/state/performance_package.rs

Description:

During the creation of a `TwapOracle`, the `aggregator` price-aggregation field is initialized to zero, and doesn't get updated until the start delay has passed.

The `price_base_performance_package` relies on the `TwapOracle` to determine whether price thresholds have been reached, and it does this by manually deserializing the aggregator's value. The code, however, doesn't special-case the state where the `aggregator` is still zero. In such a case, the final TWAP when the performance package is unlocked, will be half of the actual value, since the starting value will have been stored as zero, rather than using the `initial_observation`. This will result in performance packages not taking effect when they should.

Currently, all of the above is prevented in the launchpads' launches, by the fact that the launchpads hard-code the start delay as 24 hours, and the duration after which performance packages can initiate unlocks as 6 months, so the beginning of an unlock can never occur during the start delay.

When taken as the standalone program it is, the `price_base_performance_package` program itself is vulnerable and may affect anyone that uses it in a different way. Further, the same sort of problem can be encountered if a proposal passes with a new oracle configuration that has not passed its start delay.

The `performance_package_v2` has a similar issue, in that it gives an incorrect TWAP aggregator value prior to the start delay (the "effective aggregator"'s history will be erased once the aggregator jumps from returning zero to returning actual accumulations).

Recommendations:

Ensure that the `min_unlock_timestamp` is on or after the oracle's creation timestamp plus the oracle's start delay, for both fresh launches, and modifications of performance packages

MetaDAO: Resolved with `@824570a6fa ...` and `@de9b70b03b ...` The residual risk scenarios provided are plausible but require either non-standard integration or a governance-driven oracle change to a new, freshly-created oracle. With this in mind, we've addressed the concerns in `performance_package_v2`, as it explicitly uses a FutarchyTwap oracle. We left `price_based_performance_package` untouched, as it does not make assumptions to the kind of oracle it reads, and as such it is up to the end-user to determine whether they are locking themselves out or not. In practice, the worst case scenario is still that a single unlock cycle gets affected producing a lower TWAP reading than it should. Basically, the worst case scenario is that a team will have to run one more unlock cycle.

Zenith: Verified.

4.3 Low Risk

A total of 17 low risk findings were identified.

[L-1] `initialize_launch` omits `additional_tokens_amount` from the premine ceiling enforcement

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- initialize_launch.rs

Description

The launchpad's supply model enforces a pre-condition: premined tokens must never exceed `MAX_PREMINE` (15 000 000 × 10^6), roughly half of the combined participant-plus-liquidity allocation of 12.9M tokens. After `initialize_launch` executes, the post-condition that should hold is `total_premined ≤ MAX_PREMINE`—but the current guard only partially enforces it, leaving room for a project to mint well beyond the intended ceiling.

I traced `validate()` → the `require_gte!` block at line 158 through `initialize_launch.rs` and found that it bounds only `performance_package_token_amount`. A second premined channel—`additional_tokens_amount`—flows into the same `mint_to` call on line 305 yet is never measured against any cap. The only validation applied to `additional_tokens_amount` anywhere in the program is a presence check on its recipient:

```
require_gte!(
    MAX_PREMINE,
    args.performance_package_token_amount,
    LaunchpadError::InvalidPriceBasedPremineAmount
);

// ... further down, additional_tokens_amount is validated
// only for recipient presence, never for magnitude:
if args.additional_tokens_amount > 0 {
    require!(
        self.additional_tokens_recipient.is_some(),
        LaunchpadError::InvalidAdditionalTokensRecipient
```

```
    );
}
```

Both allocations are premined in the same `mint_to` invocation at the end of `handle()`: `performance_package_token_amount + additional_tokens_amount + TOKENS_TO_PARTICIPANTS + TOKENS_TO_FUTARCHY_LIQUIDITY + TOKENS_TO_DAMM_V2_LIQUIDITY`. Intended behavior is that the combined insider allocation stays at or below 15M tokens. Actual behavior allows it to grow without bound because `additional_tokens_amount` has zero magnitude validation—not in `initialize_launch`, nor in any other instruction across the program.

The worst case is straightforward. A project sets `performance_package_token_amount` to 10 (the enforced minimum in atoms) and `additional_tokens_amount` to 100 000 000 × 10^6 (100M tokens). Total minted supply becomes 100M + 10M + 2M + 0.9M ≈ 112.9M tokens, making the premine 100M / 112.9M ≈ 88.6% of total supply. Participants collectively hold roughly 8.9%. Even a more conservative abuse—say `additional_tokens_amount` of 25M—yields a premine of 25M / 37.9M ≈ 66%, well past the intended ~50% ceiling. After `complete_launch` finalizes the DAO, the team calls `claim_additional_token_allocation` to transfer the full uncapped amount to `additional_tokens_recipient` with no further checks (that instruction only verifies launch state and recipient match).

The practical consequence is that a project team can set `additional_tokens_recipient` to a wallet they control, pass all validation, and post-completion claim an outsized share of supply. On-chain, `performance_package_token_amount` appears modest so community members inspecting launch parameters may not realize how diluted their position truly is. The root cause is a simple omission: the state transition from uninitialized → `Initialized` does not enforce the `MAX_PREMINE` invariant over the aggregate of both premine paths.

Recommendation

Sum both premined amounts before comparing against `MAX_PREMINE`. This closes the gap with a single additive check and preserves the existing error code:

```
require_gte!(
    MAX_PREMINE,
    args.performance_package_token_amount,
    args.performance_package_token_amount
        .checked_add(args.additional_tokens_amount)
        .ok_or(LaunchpadError::InvalidPriceBasedPremineAmount)?,
    LaunchpadError::InvalidPriceBasedPremineAmount
);
```

MetaDAO: Acknowledged. At the moment, this is intentional, as we have removed MAX_PREMINE since commit `@6f2b1d8141 ...`

Note: We are always the launch authority on all Metadao and Futardio launches. We control what happens with this variable, and we always make sure it's handled in an appropriate way when there exists an addition_tokens_amount.

[L-2] Proposals cannot be finalized without trades after the proposal duration

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- futarchy/src/instructions/finalize_proposal.rs

Description:

For a proposal to either pass or fail, the TWAP since the proposal launch needs to be calculated. The code that does this uses the `last_updated_timestamp` as the "now" timestamp, with regard to the TWAP duration. This is sufficient when there is a lot of trading activity during the whole of the period, but if all of the trades occur prior to the full duration of the proposal, the market may have already reached intrinsic value, but at least one user would be forced to trade (and be charged protocol fees) in order to trigger an update to the TWAP, since there is no TWAP-cranking function.

Recommendations:

Modify `Pool::get_twap()` to project the TWAP to the current timestamp, and change the `seconds_passed` check to be based on the clock's timestamp

MetaDAO: Resolved with `@78e2b2b8e0 ...` and `@44118b6d5b ...`

Zenith: Verified.

[L-3] Possible to fund after funds have been returned

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- v06_launchpad/src/instructions/return_funds.rs

Description:

The `return_funds()` instruction allows an admin to withdraw all quote tokens from the vault, but does not change any other state in the `launch` besides the sequence number. This means that it's possible for someone to call `fund()` after `return_funds()` has been called, and to have those funds locked until the claim period.

Recommendations:

Introduce a new state, `Returned`, and change the state to this value when `return_funds()` is called.

MetaDAO: Acknowledged. This is an admin-only escape hatch instruction, and we probably won't be using it anymore as we're sunsetting the `v06_launchpad`.

[L-4] `admin_approve_execute_multisig_proposal` permits optimistic-proposal self-deadlock

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- admin_approve_execute_multisig_proposal.rs
- finalize_optimistic_proposal.rs
- initialize_proposal.rs

Description

An admin can execute the queued optimistic Squads proposal and leave `dao.optimistic_proposal` in a state that cannot be finalized or challenged, which can block future governance actions.

I traced `admin_approve_execute_multisig_proposal` to `finalize_optimistic_proposal` and then to `initialize_proposal`. The admin instruction only checks `PoolState::Spot`, and optimistic governance also runs while the AMM remains in spot mode, so the precondition is not sufficient to protect optimistic state.

```
if !matches!(self.dao.amm.state, PoolState::Spot { .. }) {
    return Err(FutarchyError::PoolNotInSpotState.into());
}
```

`finalize_optimistic_proposal` clears `dao.optimistic_proposal` only after calling `proposal_approve`. But if admin already approved/executed the same Squads proposal, that proposal is no longer `Active`, and Squads rejects approve/reject for non-active status. `initialize_proposal` also requires `Active` status, so creating a challenge proposal for cleanup fails on the same condition. A simple timing example: if `seconds_per_proposal = 86,400` and admin executes at `t0 + 90,000`, then `90,000 > 86,400`, so the optimistic window is already treated as passed while finalization still cannot clear state.

That creates a liveness failure: `dao.optimistic_proposal` persists, optimistic enqueue is blocked (`ActiveOptimisticProposalAlreadyEnqueued` path), and governance can stall until manual state surgery or a dedicated recovery path exists. This is low-likelihood because it requires admin action on a specific proposal sequence, but the outcome is high-impact because it can halt normal proposal flow.

Recommendation

Reject `admin_approve_execute_multisig_proposal` when `dao.optimistic_proposal.is_some()`, and add an admin recovery instruction that can clear optimistic state even when no futarchy `Proposal` account exists. Keep `admin_remove_proposal` for draft proposal objects, but do not rely on it for optimistic-only cleanup.

```
diff --git
    a/programs/futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs
    b/programs/futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs

     pub fn validate(&self) -> Result<()> {

         if !matches!(self.dao.amm.state, PoolState::Spot { .. }) {
             return Err(FutarchyError::PoolNotInSpotState.into());
         }

        require!(
            self.dao.optimistic_proposal.is_none(),
            FutarchyError::ActiveOptimisticProposalAlreadyEnqueued
        );

         Ok(())
     }
diff --git
    a/programs/futarchy/src/instructions/admin_clear_optimistic_proposal.rs
    b/programs/futarchy/src/instructions/admin_clear_optimistic_proposal.rs
new file mode 100644

use super::*;

#[derive(Accounts)]
pub struct AdminClearOptimisticProposal<'info> {
    #[account(mut)]
    pub dao: Box<Account<'info, Dao>>,
    pub admin: Signer<'info>,
}

impl AdminClearOptimisticProposal<'_> {
    pub fn validate(&self) -> Result<()> {
        #[cfg(feature = "production")]
        require_keys_eq!(self.admin.key(), admin::ID, FutarchyError::
```

```
            InvalidAdmin);

        require!(self.dao.optimistic_proposal.is_some(), FutarchyError::
            NoActiveOptimisticProposal);
        Ok(())
    }

    pub fn handle(ctx: Context<Self>) -> Result<()> {
        let dao = &mut ctx.accounts.dao;
        dao.optimistic_proposal = None;
        dao.seq_num += 1;
        Ok(())
    }
}
```

MetaDAO: Resolved with PR-396.

Zenith: Verified.

[L-5] Proposals may use stale proposal durations

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- futarchy/src/instructions/initialize_proposal.rs

Description:

During `Proposal` creation, the current value of the `Dao.seconds_per_proposal` is stored to the `Proposal` as the `duration_in_seconds`. If the `Dao` later changes its duration, this update does not get applied to existing proposals, even if they are many years old. In contrast, optimistic proposals do not store their own durations and instead rely on the live value. A team could queue up thousands of normal token transfer squads proposals, and if the proposal duration ever moves away from their favor for optimistic spends, they can use one of the old `Proposals` to shorten the duration back to what it was.

Recommendations:

Don't store the duration until the `Proposal` is launched

MetaDAO: Resolved with `@b90932e5ee ...`

Zenith: Verified.

[L-6] Users can still stake to invalidated transactions

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- futarchy/src/instructions/stake_to_proposal.rs

Description:

Unlike is the case for initialization of proposals, staking to proposals does not verify that the transaction has not yet been invalidated. If the Squads proposal was created and used to initialize a `Proposal` prior to a configuration execution that changes the stale transaction threshold, it would be possible for users to stake to a `Proposal` that is ultimately unable to be launched, diluting staking power that could have been concentrated elsewhere.

Recommendations:

Add a threshold check to the `stake_to_proposal()` ix

MetaDAO: Acknowledged. A fix requires us to expand the account surface of the stake instruction with little to no realistic gain, as the stake can be unstaked without any blockers. The relevant instructions - `initialize_proposal` and `launch_proposal` already have the necessary staleness checks which prevents launching proposals that point to stale squads proposals.

[L-7] A DAO's team can prevent legitimate proposals by continuously sponsoring and launching proposals

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- futarchy/src/instructions/sponsor_proposal.rs

Description:

If there is huge support for a proposal that the DAO's team is against, the team can continuously DOS it by creating, sponsoring, and launching proposals that they know will fail. Alternatively, the team can continuously launch the same sort of unpopular proposal until it passes with the lower threshold, or may repeat the same optimistic proposals. The team does not have to lock up tokens, and there is no cooldown between sponsorings.

Recommendations:

Add a shared cooldown to team invocations of `sponsor_proposal()` and `initiate_vault_spend_optimistic_proposal()`, so that share holders can launch proposals to turn off optimistic spends, and increase the team sponsor threshold

MetaDAO: Acknowledged, and we're aware of it. For now, we won't be adding additional mechanics (such as the cooldown) before we see this becoming an issue, as we can intervene (for now). Such a situation would deeply erode trust in the DAO, and while possible, would essentially end up in a liquidation of the DAO.

[L-8] DAO initialization checks from v07 missing in v06

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- v06_launchpad/src/instructions/initialize_launch.rs
- v07_launchpad/src/instructions/initialize_launch.rs

Description:

In the v7 version of the code, the `monthly_spending_limit_members` is Verified. to be non-empty and to not have any duplicates, since this will fail the creation of the multisig. The v6 code is missing these checks and still has the `MAX_PREMINE` check that was removed in v07.

Recommendations:

Ensure the same checks are done in both versions

MetaDAO: Resolved with `@27d5517447 ...`

Zenith: Verified. The members-related checks have been added to v6, which still has the `MAX_PREMINE` check.

[L-9] DAO initialization ignores the user's TWAP start delay for the spot market

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- futarchy/src/instructions/initialize_dao.rs

Description:

In the `initialize_dao()` instruction, the DAO stores the input `twap_start_delay_seconds` for use when launching proposals and their pools, but when it comes to the initial spot market, the instruction hard-codes the delay to zero. This may cause external consumers of the spot market's price (e.g. performance package v2), to consume insecure TWAP prices.

Recommendations:

```
    twap_max_observation_change_per_update,
  0,
  twap_start_delay_seconds,
),
```

MetaDAO: Acknowledged. This is intended, as the TWAP start delay is a conditional market-specific mechanism. Regarding external readers, the initial observation is set correctly when the DAO is launched through a launchpad. While the `aggregator` may be zero if there have not been any trades, it's the reader's responsibility to take a look at `(aggregator, observation, last_updated)` and project the correct aggregator value when reading the aggregator value, and all 3 are available to them.

[L-10] Performance packages may lose dust amounts due to rounding

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- v06_launchpad/src/instructions/initialize_launch.rs
- v07_launchpad/src/instructions/initialize_launch.rs
- v06_launchpad/src/instructions/complete_launch.rs
- v07_launchpad/src/instructions/initialize_performance_package.rs

Description:

One of the arguments to launch initialization is the amount of tokens to put towards the performance package. The only constraint on the amount is that it's at least ten atomic units. Later, when the performance package is being created, this amount is divided into five tranches. If the amount provided in during launch initialization is not an exact multiple of five, each of the tranches will contain fewer tokens than expected, since dividing the amount by five throws away the remainder, due to integer division. After the whole performance package period has elapsed, the dust amount of tokens will remain stuck in the performance package's vault.

Recommendations:

Add the remainder to the last tranche

MetaDAO: Acknowledged, but not worth fixing at this time, due to us moving to PPv2 and a Mint Governor system. Our launch mechanisms currently ensure that all MetaDAO-launched projects have these numbers set up correctly.

[L-11] The `execute_change()` instructions return rent to the wrong recipient

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- price_based_performance_package/src/instructions/execute_change.rs
- performance_package_v2/src/instructions/execute_change.rs

Description:

The final step of executing a performance package change, is to close the PDA that was created by the proposer, and to send the rent to an address under the executor's control. The executor is intentionally restricted to *not* be the party that created the PDA in the first place, so it is incorrect for the executor to decide where the PDA's rent gets sent.

Recommendations:

Have a separate instruction for closing proposals by the creator

MetaDAO: Acknowledged, but it ultimately doesn't matter since we could even have a situation where we have 2 live change proposals which would create the rent going into the wrong hands. Won't fix for now, as it adds unnecessary complexity to a low-usage instruction.

[L-12] Performance packages' change requests ignore package re-creations

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- price_based_performance_package/src/instructions/burn_performance_package.rs
- performance_package_v2/src/instructions/close_performance_package.rs

Description:

When a performance package of either version is closed/burned and re-created, prior `ChangeRequests` that have been approved but not yet executed are not invalidated. If the oracle of the second incarnation is different from the change request's, the recipient could confirm an oracle change that only made sense in the context of the prior tranche layout, which could immediately give them access to tokens.

A similar issue is that it's possible for a recipient to propose a change, for a separate proposal to change the recipient, and then for the admin to approve the change that was only agreed to by the old recipient. An analogous issue exists with the admin approving a change, the admin changing, then the recipient executing the prior admin's proposal.

Recommendations:

Have a last-updated slot value in the performance package and store it in the `ChangeRequest`, and confirm that they match during execution

MetaDAO: Resolved with `@9bab52dd71 ...` and `@4ecc93c49f ...` Going with the timestamp of last update would invalidate change requests that were created in parallel.However, we decided to record when the PP was created. This gives me a "version" of the PP, and the change request knows which "version" it targets, thus it can be deemed stale if it doesn't target the same version. Another thing we had to do for this is also ensure that both parties gave explicit consent, meaning that the proposer musn't have changed. Added that too.

Zenith: Verified. The `price_based_performance_package` issue is acknowledged.

[L-13] No upper limit on performance package v2's unlock window

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- performance_package_v2/src/state/performance_package.rs

Description:

Unlike `price_based_performance_package`, which has an upper bound on the duration of the unlock window, `min_duration`. If the window is too large and an unlock has begun, there is no way to cancel the unlock in order to allow for it to be modified or admin-closed

Recommendations:

Add an upper bound to the `min_duration`

MetaDAO: Resolved with `@8f4a2aa70e ...`

Zenith: Verified.

[L-14] Events may be dropped if the SVM truncates its logs

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- mint_governor/src/instructions/**/*.rs
- performance_package_v2/src/instructions/**/*.rs

Description:

If there are other log-heavy instructions prior to ones in `mint_governor` and `performance_package_v2`, the SVM logs may get truncated, causing the events to be silently dropped, disrupting off-chain tracking for these programs. Since there are some instructions in `price_based_performance_package` that do CPI emissions, it is likely that analogous instructions, at least, would need to also do CPI emissions. Note also that many of the instructions in `price_based_performance_package` do normal emissions rather than the CPI versions.

Recommendations:

Use `emit_cpi!()` for events that require off-chain tracking

MetaDAO: Resolved with `@a35dfe1071 ...`

Zenith: Verified.

[L-15] Strict PDA derivations may require client programs to re-deploy

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- mint_governor/src/instructions/initialize_mint_governor.rs
- performance_package_v2/src/instructions/initialize_performance_package.rs

Description:

In the case of the `mint_governor` program, the seeds of a `MintGovernor` PDA limits instances to one per `create_key` signer: `[MINT_GOVERNOR_SEED, mint.key().as_ref(), create_key.key().as_ref()]`. One of the configuration parameters is an `admin` that does not need to be a signer of the creation transaction. If the program wishing to integrate with the `mint_governor` program has a typo in their `admin` Pubkey, there is no way to create a new `MintGovernor`, without re-deploying the program at a new `create_key`, which may be infeasible.

The `performance_package_v2` program has a similar issue in that the seeds of a `PerformancePackage` PDA limits instances to one per `create_key` signer: `[PERFORMANCE_PACKAGE_SEED, create_key.key().as_ref()]`, and requires `recipient` that does not need to be a signer. This case is less severe because there is a `close_performance_package()` instruction that can be used by an admin to destroy a misconfigured `PerformancePackage`

Recommendations:

In the case of `MintGovernor`, require the `admin` to be a `Signer`, and for both programs, include a nonce in the PDA seed so that multiple can be created.

MetaDAO: Acknowledged. Unfortunately, `admin` cannot be a signer in this case as it would imply that our launchpad can arbitrarily invoke Squads transactions. The intended use-case is for a Squads multisig vault to be the admin of a Mint Governor account.The `create_key` is required to be a signer specifically to prevent others from front-running creation of accounts where `create_key` would be known, such as when we complete a launch. It also allows us to generate a random keypair, sign the tx, and throw it away, thus making it impossible for a new PP to be created with the same `create_key`.

Nonce won't matter in our case, as a single launch only ever creates a single PP. We aren't that concerned about typo-related footguns.

[L-16] Bid wall closure may send ATA rent to wrong recipient

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- bid_wall/src/instructions/close_bid_wall.rs

Description:

The closure of bid walls is done via a permissionless instruction. The `BidWall`'s PDA rent properly gets sent to the bid wall authority, but the rent associated with the bid wall's quote ATA gets sent to a signer under the control of the caller of the instruction.

Recommendations:

Send the quote ATA's rent to the bid wall's authority too

MetaDAO: Resolved with `@25226e2a2f ...`

Zenith: Verified.

[L-17] Early `start_unlock()`s by the `performance_package_v2` authority may DOS recipient payouts

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- performance_package_v2/src/instructions/start_unlock.rs

Description:

Unlike the v1 performance package that only allows the recipient to initiate an unlock, the v2 code allows the performance package's authority to initiate an unlock. The completion of the unlocking operation requires that the end of the unlock occur on or after the end of the of the TWAP duration, which currently has no upper limit. If the package's authority prematurely initiates an unlock, when the spot price has not yet reached any of the package's thresholds, the recipient will have to wait the full TWAP duration to complete the failed unlock (assuming that the earliness ends up lowering the ending total TWAP value to below the threshold as well), then re-initiate a new unlock, which also may take an extremely long time. This effectively DOS the recipient's payout for an extra TWAP duration.

Recommendations:

Allow the recipient to reset the unlock

MetaDAO: Resolved with `@f6b21d219c ...`

Zenith: Verified.

4.4 Informational

A total of 9 informational findings were identified.

[I-1] `FundingRecords` incompletely Verified.

`SEVERITY`: Informational	`IMPACT`: Informational
`STATUS`: Resolved	`LIKELIHOOD`: Low

Target

- v07_launchpad/src/instructions/claim.rs
- v06_launchpad/src/instructions/claim.rs
- v07_launchpad/src/instructions/refund.rs
- v06_launchpad/src/instructions/refund.rs

Description:

Both the `Claim.funding_record` and the `Refund.funding_record` accounts are Verified. by their PDA derivations via their seeds and bumps:

```
#[account(
    mut,
    has_one = launch_quote_vault,
    has_one = launch_signer,
)]
pub launch: Account<'info, Launch>,

#[account(
    mut,
    has_one = funder,
    seeds = [b"funding_record", launch.key().as_ref(),
    funder.key().as_ref()],
    bump = funding_record.pda_bump
)]
pub funding_record: Account<'info, FundingRecord>,
```

While it's almost impossible for two PDAs with different seeds to collide, the probability is not 100%. In the future there may be a weakness found in SHA256 which makes it

vulnerable to second pre-image attacks. In such a scenario, it would be possible for an attacker to grind launches such that one `funder` applies to two `launches` (and may have same bump). They would be able to create a colliding `launch` with a raise amount matching the amount to fund and steal, get refunded from the victim launch, and get paid back as a DAO by the second launch, via runway payments.

Note that neither MD5 nor SHA1 have had second preimage resistance broken, so it is extremely unlikely for SHA256 to be broken, so this issue is currently purely theoretical.

Recommendations:

Remove the `seeds/bump`-based check, and replace it with a `has_one = launch`

MetaDAO: Resolved with `@f0f97d7173 ...` I'm a bit more comfortable with adding both a seeds check and a constraint, so I added the constraint in addition to the seed check.

Zenith: Verified.

[I-2] Accounts need not be marked as mutable

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- v07_launchpad/src/instructions/claim_additional_token_allocation.rs
- v07_launchpad/src/instructions/claim.rs
- v06_launchpad/src/instructions/claim.rs
- futarchy/src/instructions/finalize_optimistic_proposal.rs
- futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs
- futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs
- futarchy/src/instructions/admin_cancel_proposal.rs
- price_based_performance_package/src/instructions/propose_change.rs
- bid_wall/src/instructions/sell_tokens.rs
- v07_launchpad/src/instructions/initialize_performance_package.rs

Description:

The following accounts need not be mutable since the instructions don't modify the accounts' state.

- `ClaimAdditionalTokenAllocation.launch_signer`
- `ClaimAdditionalTokenAllocation.base_mint`
- `Claim.base_mint` (v06/v07)
- `FinalizeOptimisticProposal.squads_multisig`
- `AdminApproveExecuteMultisigProposal.squads_multisig`
- `AdminApproveExecuteMultisigProposal.squads_multisig_vault_transaction`
- `AdminCancelProposal.admin`
- `ProposeChange.performance_package`
- `SellTokens.user`
- `InitializePerformancePackage.launch_signer`

This will lead to unnecessary transaction contention, possibly leading to higher transaction costs required for the instructions.

Recommendations:

Remove the `mut` modifier from the listed accounts

MetaDAO: Resolved with `@4d543caa93 ...`

Zenith: Verified.

[I-3] Missing events in some instructions

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs
- futarchy/src/instructions/execute_spending_limit_change.rs
- price_based_performance_package/src/instructions/burn_performance_package.rs
- price_based_performance_package/src/instructions/propose_change.rs

Description:

The accounts struct `AdminApproveExecuteMultisigProposal`, for the `admin_approve_execute_multisig_proposal()` instruction uses the `#[event_cpi]` attribute macro, implying that it intends to do CPI event emissions like the other admin-related instructions do. However, there are no events emitted by the `handle()` method.

The instruction handling related to `ExecuteSpendingLimitChange`, `BurnPerformancePackage`, and `ProposeChange` have the same issues.

Recommendations:

Define and CPI-emit an event

MetaDAO: Resolved with `@25abdade0a ...`, `@25226e2a2f ...` and `@d17e46dcf2 ...` `admin_approve_execute_multisig_proposal` was split into two IXes, but as they are admin-only, we will not be adding events to those.

Zenith: Verified.

[I-4] Squads `TransactionMessage` compilation does not match Squads SDK

`SEVERITY`: Informational	`IMPACT`: Informational
`STATUS`: Resolved	`LIKELIHOOD`: Low

Target

- programs/futarchy/src/squads.rs

Description:

In the Squads SDK, the first writable signer is set to be the signing vault itself, whereas the MetaDAO code assigns it as the first readonly signer. On the Squads program side, the code ignores the signer-ness, but keeps the account as writable.

There currently are no negative effects, because the `compile_squads_transaction_message()` function is not being used to spend the vault's SOL. If it were to be used in that way, it would fail.

Recommendations:

Update the function to match the SDK exactly

MetaDAO: Resolved with `@c0b75aaa25 ...`

Zenith: Verified.

[I-5] Funding is allowed for one second longer than expected

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- v07_launchpad/src/instructions/fund.rs
- v06_launchpad/src/instructions/fund.rs
- v07_launchpad/src/instructions/set_funding_record_approval.rs
- futarchy/src/state/futarchy_amm.rs
- price_based_performance_package/src/instructions/initialize_performance_package.rs

Description:

The funding period validation uses `require_gte!()` ensure that funds can only be contributed during the live period:

```
require_gte!(
    self.launch.unix_timestamp_started.unwrap() +
    self.launch.seconds_for_launch as i64,
    clock.unix_timestamp,
    LaunchpadError::LaunchExpired
);
```

If the `seconds_for_launch` were set to zero (not currently allowed by the validations during initialization), funding would be allowed during the second in which the launch went live. This means that one second of launch was fundable rather than zero.

The setting of the funding amount check has a similar issue, as does the TWAP's start timestamp check. The v1 performance package has the inverse issue (v2 does the correct check).

Recommendations:

Modify the require statements to include the current second

MetaDAO: Resolved the funding issues `@6b4c5b6aee ...`, `@25226e2a2f ...` and `@8f8b090921 ...` `futarchy_amm.rs` — At the boundary second, `time_difference` is always 0 so accumulation is a no-op regardless of ≤ vs <.

Zenith: Verified. The `futarchy_amm.rs` issue is acknowledged.

[I-6] `OracleReader::can_end()` reports the wrong answer if `record_start()` has not been called

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- performance_package_v2/src/state/performance_package.rs

Description:

If `OracleReader::record_start()` has not been called prior to calling `can_end()`, the `start_time` used will be zero:

```
match clock {
    Ok(clock) => {
        let elapsed = clock.unix_timestamp.saturating_sub(*start_time);
        elapsed >= *min_duration as i64
    }
    Err(_) => false,
}
```

The elapsed time will be reported as the total number of seconds since 1970, and this will likely exceed the `min_duration`. No code is currently doing this, but this should be prevented for future changes.

Recommendations:

Return `false` if `*start_time` is zero

MetaDAO: Resolved with `@5962fcefd7 ...`

Zenith: Verified.

[I-7] Unused instruction accounts

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- v07_launchpad/src/instructions/fund.rs
- v06_launchpad/src/instructions/fund.rs
- v07_launchpad/src/instructions/refund.rs
- v06_launchpad/src/instructions/refund.rs
- v07_launchpad/src/instructions/claim.rs
- v06_launchpad/src/instructions/claim.rs
- v06_launchpad/src/instructions/return_funds.rs
- futarchy/src/instructions/stake_to_proposal.rs
- futarchy/src/instructions/unstake_from_proposal.rs
- performance_package_v2/src/instructions/complete_unlock.rs
- bid_wall/src/instructions/collect_fees.rs
- bid_wall/src/instructions/sell_tokens.rs#L49

Description:

The following accounts are not used by the respective instructions, which unnecessarily increases transaction costs:

- `Fund.launch_signer` (v06/v07)
- `Refund.system_program` (v06/v07)
- `Claim.system_program` (v06/v07)
- `ReturnFunds.system_program`
- `StakeToProposal.associated_token_program`
- `UnstakeFromProposal.associated_token_program`
- `CompleteUnlock.associated_token_program`
- `CollectFees.system_program`
- `SellTokens.system_program`

Recommendations:

Remove the unused accounts

MetaDAO: Resolved with `@eaf2662e55 ...`

Zenith: Verified.

[I-8] Misleading comments

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- v07_launchpad/src/lib.rs
- v06_launchpad/src/lib.rs
- v07_launchpad/src/instructions/initialize_launch.rs
- v06_launchpad/src/instructions/initialize_launch.rs
- v07_launchpad/src/instructions/initialize_launch.rs
- v06_launchpad/src/instructions/initialize_launch.rs
- v07_launchpad/src/instructions/complete_launch.rs
- v06_launchpad/src/instructions/complete_launch.rs
- programs/futarchy/src/state/dao.rs
- futarchy/src/state/dao.rs
- futarchy/src/instructions/conditional_swap.rs
- price_based_performance_package/src/events.rs
- futarchy/src/instructions/collect_meteora_damm_fees.rs

Description:

The following comments are misleading, which adds unnecessary cognitive load during reviews:

- The `lib.rs` of both `v06_launchpad` and `v07_launchpad` define constants related to hard-coded token amounts. It appears that at one point 3M tokens were provided as single-sided DammV2 liquidity, which means the sum of the non-premine tokens was 15M tokens, and all percentages were based off of this number. Later, 3M was changed to 900K, but the comment for `TOKENS_TO_DAMM_V2_LIQUIDITY` and none of the other percentages were updated. Note that the error message for `InvalidPriceBasedPremineAmount` would need to be updated as well:

```
/// 10M tokens with 6 decimals
pub const TOKENS_TO_PARTICIPANTS: u64 = 10_000_000 * TOKEN_SCALE;
/// 20% to liquidity
pub const TOKENS_TO_FUTARCHY_LIQUIDITY: u64 = 2_000_000 * TOKEN_SCALE;
/// 3M tokens to single-sided DammV2 liquidity
```

```
pub const TOKENS_TO_DAMM_V2_LIQUIDITY:
    u64 = TOKENS_TO_DAMM_V2_LIQUIDITY_UNSCALED * TOKEN_SCALE;
/// we need this to prevent overflow
pub const TOKENS_TO_DAMM_V2_LIQUIDITY_UNSCALED: u64 = 900_000;
/// 15% of the floating supply to stake
pub const PROPOSAL_MIN_STAKE_TOKENS: u64 = 1_500_000 * TOKEN_SCALE;

/// Max 50% premine
pub const MAX_PREMINE: u64 = 15_000_000 * TOKEN_SCALE;
```

- In the `InitializeLaunch::validate()` functions of both `v06_launchpad` and `v07_launchpad`, there are checks for the miminum launch duration that are commented out. If there needs to be a minimum duration, the code should be re-enabled. Otherwise, the commented out code should be removed:

```
// #[cfg(not(feature = "devnet"))]
// require_gte!(
//     args.seconds_for_launch,
//     60 * 60,
//     LaunchpadError::InvalidSecondsForLaunch
// );
```

- In the `InitializeLaunch::validate()` functions of both `v06_launchpad` and `v07_launchpad`, there are also checks that the `Pubkey` ends with "meta", that are commented out. This also causes `LaunchpadError::InvalidTokenKey` to become an unused error code:

```
// #[cfg(feature = "production")]
// {
//     let base_token_key:  String = self.base_mint.key().to_string();
//     let last_4_chars = &base_token_key[base_token_key.len() - 4..];
//     require_eq!("meta", last_4_chars, LaunchpadError::InvalidTokenKey);
// }
```

- The comments mention having to lock up liquidity in pass/fail markets in order to create a proposal, but there is no such functionality; stakes can be reclaimed at any point, and the liquidity comes from the spot market instead

```
/// As an anti-spam measure and to help liquidity, you need to lock up some
    liquidity
/// in both futarchic markets in order to create a proposal.
///
/// For example, for META, we can use a `min_quote_futarchic_liquidity` of
/// 5000 * 1_000_000 (5000 USDC) and a `min_base_futarchic_liquidity` of
```

```
/// 10 * 1_000_000_000 (10 META).
```

```
// For the DAO, we want proposals to start at the price of the launch,
// for the lagging TWAP to be able to move its latest observation by 5%
// per update (300% per hour), and for proposers to need to lock up 1%
// of the supply and an equivalent value of USDC.
```

- The comment says that the ATAs are checked in `validate()`, but there are no such checks in that method. Instead, all validations are done by Anchor account constraints

```
// These are checked in `validate`
#[account(mut, associated_token::mint = proposal.pass_base_mint,
    associated_token::authority = dao)]
pub amm_pass_base_vault: Box<Account<'info, TokenAccount>>,
#[account(mut, associated_token::mint = proposal.pass_quote_mint,
    associated_token::authority = dao)]
pub amm_pass_quote_vault: Box<Account<'info, TokenAccount>>,
#[account(mut, associated_token::mint = proposal.fail_base_mint,
    associated_token::authority = dao)]
pub amm_fail_base_vault: Box<Account<'info, TokenAccount>>,
#[account(mut, associated_token::mint = proposal.fail_quote_mint,
    associated_token::authority = dao)]
pub amm_fail_quote_vault: Box<Account<'info, TokenAccount>>,
```

- The event's field is commented out, and it's not clear whether this was a temporary test, or whether the field is purposely being removed

```
pub struct PerformancePackageInitialized {
    pub common: CommonFields,
    pub performance_package: Pubkey,
    // TODO: see CU gain of not including this
    // pub performance_package_data: PerformancePackage,
}
```

- The non-Squads signer is required to prevent Squads reentrancy

```
// We must use a non-Squads signer because of CPI depth limits
declare_id!("tSTp6B6kE9o6ZaTmHm2ZwnJBBtgd3x112tapxFhmBEQ");
```

Recommendations:

Update the comments to reflect the current behavior.

MetaDAO: Acknowledged.

[I-9] Unused account fields waste rent

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- v07_launchpad/src/state/launch.rs
- v06_launchpad/src/state/launch.rs
- performance_package_v2/src/state/change_request.rs
- price_based_performance_package/src/state/performance_package.rs

Description:

The `Launch` structs contains a `pda_bump` member for tracking the PDA's bump. Once the member is initialized, the bump is never used, and thus wastes both rent and CUs. There is a similar issue with `ChangeRequest.pda_nonce` and `ChangeRequest.pda_bump`.

Recommendations:

Remove the fields from the listed structs, and the code that sets their values.

MetaDAO: Acknowledged, won't fix for now. While it's a neat space-saving measure, these are intentionally stored to simplify future work where these might be needed. Since these are not the final versions/revisions of our programs, we'd prefer to keep the extra fields around until we decide that optimization matters.